HATHOR-CORE · MASTER-BOOK

Python the Platform

Not the syntax you already know — the machinery beneath it: the interpreter that runs your code, the modules that organize it, and the import system that stitches a thousand files into one program.

SUBJECT	hathor-core · Part I · Track C (the stack)
CHAPTER	11 · Foundations · The Stack
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

CPython · Bytecode · Interpreter · REPL · Modules · Packages · __init__.py · Imports · sys.path · sys.modules · GIL

Table of Contents

Chapter 11 — Python the Platform

What you'll learn in this chapter

- That "Python" is three things, not one: a language, an interpreter that runs it (**CPython**), and an ecosystem of modules and tools — and why a study of `hathor-core` cares about the latter two.
- What the interpreter actually does with your file: source → **bytecode** → execution, and what the **REPL** is.
- How Python organizes code: a file is a **module**, a directory is a **package**, and what `__init__.py` is for.
- How `import` truly works — the search path (`sys.path`), the module cache (`sys.modules`), and the difference between absolute and relative imports — so a line like `from hathor.transaction import Block` stops being magic.
- **Why Python** for a node at all, given it is not the fastest language — the trade-off, and how Hathor mitigates the cost.
- A **bridge** to the rest of Track C and to where the codebase's structure relies on these mechanics.

Track A and Track B taught concepts. Track C turns to the **stack** — the third-party technologies and platform machinery `hathor-core` is built from — and it starts at the bottom, with Python itself. Not the syntax (you have that), but the platform: the interpreter that executes the code, and the module-and-import system that lets one program span hundreds of files across dozens of packages. You cannot reason about how `hathor-core` is assembled, packaged, or run without these, and every later Track C chapter (virtual environments, Poetry, Docker) sits directly on top of them.

This chapter follows the §5 technology-primer shape: what it is, the problem it solves, the core model, how you use it, how Hathor relies on it, and the "why this and not something else" trade-off.

11.1 "Python" is three things

When people say "Python," they conflate three distinct things, and separating them is the first step to understanding the platform:

1. **The language** — the rules of syntax and semantics: what `for`, `def`, `class`, and an f-string mean. This is what you already know and what a language specification defines.

2. **The interpreter** — the actual program, installed on your machine, that reads Python source and executes it. The standard one is **CPython**[1], written in C. When you type `python`, you are running CPython. There are others (PyPy, Jython), but CPython is what `hathor-core` runs on.
3. **The ecosystem** — the standard library that ships with the interpreter, plus the vast collection of third-party packages (Twisted, RocksDB bindings, Pydantic…) installed from outside, plus the tools that manage all of it (pip, Poetry).

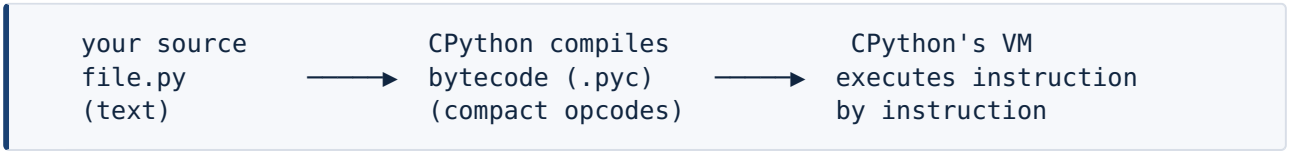
The language is the same everywhere; the interpreter and ecosystem are where the platform lives. A node is a large program leaning heavily on the third item (dozens of external packages) and constrained by the second (CPython's performance characteristics, including the GIL from Chapter 2). So Track C is, in effect, a tour of items 2 and 3.

11.2 The interpreter: what `python file.py` actually does

A common misconception is that Python is "interpreted line by line" like reading a script aloud. The reality is a two-step process, and knowing it explains several things you'll see in the repo.

When you run `python file.py`, CPython:

1. **Compiles** the source to **bytecode**[2] — a lower-level, compact instruction set that is not machine code for your CPU, but instructions for Python's own virtual machine. This is a compile step, even though Python is called "interpreted."
2. **Executes** that bytecode on the **Python Virtual Machine**[3] — a loop inside CPython that reads bytecode instructions one at a time and performs them.



Two visible consequences:

- **`__pycache__` and `.pyc` files.** You'll see directories named `__pycache__` full of `.pyc` files appear throughout a project after it runs. These are CPython caching the compiled bytecode so it needn't recompile unchanged files next time — a speed optimization, nothing you author or commit. (They're in every project's `.gitignore` for that reason.)
- **"Compiled" but not to machine code.** Unlike C or Rust, the bytecode still runs on the VM, not directly on the processor. That indirection is part of why

Python is slower than those languages (§11.6) — and why the bytecode is portable across operating systems.

The REPL. Run `python` with no file and you get the **REPL**[4] — Read-Eval-Print Loop — an interactive prompt that compiles and runs each line as you type it, printing results immediately. It is the fastest way to poke at behavior, and `hathor-core` even ships a subcommand (`hathor-cli shell`, Chapter 21) that drops you into a REPL with the node's objects loaded — invaluable for exploring the running system.

11.3 Modules: a file is a unit of code

A program of any size cannot live in one file. Python's unit of organization is the **module**[5]: a single `.py` file is a module, and its name is the filename without the extension. The file `daa.py` is the module `daa`.

A module is more than a container — it is a **namespace**[6]. Everything defined at the top level of `daa.py` (its functions, classes, constants) becomes an attribute of the module object, reached with a dot: `daa.DifficultyAdjustmentAlgorithm`. This is the same dot-access you met on objects in Chapter 1, applied to files. Importing a module runs its top-level code once and hands you that namespace.

That "once" matters and is worth fixing now: **a module's top-level code executes exactly once per program, the first time it is imported.** Import it again from somewhere else and you get the already-loaded module back, not a fresh run (§11.5 explains the cache that guarantees this). This is why module-level code is the natural home for things that should exist once — a logger, a settings accessor, a registry — a fact that connects directly to the singleton pattern of Chapter 3.

11.4 Packages: a directory is a unit too

Modules group code within a file; **packages**[7] group modules across directories. A package is a directory of modules treated as one importable unit. So `hathor/` is a package, `hathor/transaction/` is a sub-package, and `hathor/transaction/block.py` is the `block` module inside it. The dotted path `hathor.transaction.block` mirrors the folder path `hathor/transaction/block.py` exactly — packages are how the filesystem layout becomes the import namespace.

The traditional marker of a package is a file named `__init__.py` in the directory. It does two jobs:

1. **It declares "this directory is a package"** (in classic Python; modern "namespace packages" can omit it, but `hathor-core` uses explicit `__init__.py` files).

2. **It runs when the package is first imported**, so it's a place to set up package-level state or to re-export names. You saw exactly this in Chapter 22: `hathor/conf/__init__.py` re-exports `HathorSettings` so callers can write the short `from hathor.conf import HathorSettings` instead of reaching into the deeper module. An `__init__.py` that gathers a package's public names into one tidy surface is a common, deliberate pattern.

```
hathor/                     ← package (has __init__.py)
├── __init__.py
├── manager.py              ← module  hathor.manager
├── conf/                   ← sub-package
│   ├── __init__.py         ← re-exports HathorSettings (Ch 22)
│   └── settings.py         ← module  hathor.conf.settings
└── transaction/            ← sub-package
    ├── __init__.py
    └── block.py            ← module  hathor.transaction.block
```

The dotted name and the directory tree are the same structure seen two ways. Once you internalize that, the whole repository becomes navigable from any import line: `from hathor.transaction.block import Block` tells you exactly which file to open.

11.5 How `import` actually works

Import is where beginners hit "magic," so we make it concrete. When Python executes `import hathor.transaction.block`, it does three things, in order:

1. Check the cache first. Python keeps a dictionary of every module already imported, `sys.modules`[8], keyed by dotted name. If `hathor.transaction.block` is already there, Python returns it instantly and runs nothing further. This is the mechanism behind "top-level code runs once" (§11.3) — the first import populates the cache; every later import is a cache hit. It also means circular imports (A imports B which imports A) are a real hazard, because the second import may find a half-initialized module in the cache.

2. Find the module. On a cache miss, Python searches a list of directories called `sys.path`[9] — an ordered list of places to look, including the current directory, the standard library, and the installed-packages location (the `site-packages` of your environment, which is Chapter 12's subject). The first match wins. This search path is exactly what a virtual environment manipulates to make a project see its own dependencies and not the system's.

3. Load, execute, cache. Once found, Python compiles the module to bytecode (§11.2), executes its top-level code to build the namespace, stores it in `sys.modules`, and binds the name in your code.

```
import hathor.transaction.block
     │
     ▼
in sys.modules?  ──yes──▶  return cached module (run nothing)
     │ no
     ▼
search sys.path for it  ──▶  compile → run top-level → store in sys.modules → bind
```

Absolute vs. relative imports. Two spellings reach a module:

- **Absolute**: `from hathor.transaction.block import Block` — the full path from the project root. Unambiguous, readable, and the dominant style in `hathor-core`.
- **Relative**: `from .block import Block` or `from ..conf import HathorSettings` — the leading dots mean "relative to this package" (one dot = current package, two = parent). Shorter inside a package, but only usable within a package. You'll see both; the absolute form tells you the file outright, which is why this book cites with full paths.

That is the entire import system: a cache, a search path, and a load step. No magic — just a disciplined way to turn a dotted name into a found, compiled, cached namespace.

11.6 Why Python for a node?

A full node is performance-sensitive infrastructure that runs for months and verifies cryptographic data constantly. Python is not the fastest language — CPython's bytecode VM and the GIL (Chapter 2) make pure-Python computation far slower than C, Rust, or Go. So the choice deserves the §5 "why this, not the alternatives" treatment.

What Python costs: raw CPU speed. A tight numeric loop in Python can be one to two orders of magnitude slower than the equivalent in C. For a CPU-bound program, that would be disqualifying.

Why it's chosen anyway:

- **A node is I/O-bound, not CPU-bound (Chapter 2 §2.1).** It spends its life waiting on the network and disk, not crunching numbers. For waiting-dominated work, the language's raw speed barely matters; the concurrency model matters, and Twisted (Chapter 16) gives Python an excellent one.
- **Development speed and clarity.** Python is fast to write, read, and change — valuable for a complex, evolving protocol where correctness and auditability beat microseconds. A node's bugs are expensive; readable code that many people can review is a real asset.

- **A deep ecosystem.** The exact libraries a node needs already exist and are mature: Twisted for async networking, battle-tested crypto bindings, RocksDB bindings, Pydantic. Building on these beats reimplementing them.
- **The slow parts are pushed into C.** The trick that makes the trade work: the genuinely CPU-heavy pieces don't run in Python. RocksDB is C++ (Chapter 27); the cryptography library wraps C (Chapter 40); proof-of-work hashing runs in a thread pool over compiled hash routines (Chapter 2 §2.7, Chapter 37). Python orchestrates; compiled code does the number-crunching. You get Python's clarity at the top and near-C speed where it counts.

The honest summary: Python trades raw speed for development velocity and ecosystem, and a node can afford that trade because it is I/O-bound and because its hot paths delegate to compiled libraries. Pick Go or Rust and you'd gain CPU speed and lose iteration speed and the specific mature libraries Hathor leans on — a different, also-defensible choice that other node implementations make. (The version is pinned, too: `hathor-core` requires a modern Python — the exact constraint lives in `pyproject.toml`, Chapter 13.)

11.7 Bridge — the platform under the codebase

Everything above is the substrate the rest of the book stands on. Forward-pointers:

Bridge — the Python platform in the codebase and the stack:

- **The import tree IS the architecture.** Every `from hathor.x.y import Z` in this book maps to the file `hathor/x/y.py` (§11.4). The module map of Chapter 0 is, literally, the package tree — **Chapter 0**, and every Part II chapter.
- **`__init__.py` re-exports.** The package-surface pattern (§11.4) is used deliberately, e.g. `hathor/conf/__init__.py` — **Chapter 22**.
- **Module-level singletons.** "Top-level code runs once" (§11.3, §11.5) underlies the global settings accessor and the one-per-process reactor — **Chapters 22, 16, 23** (and the singleton pattern, Chapter 3).
- **`sys.path` is what environments manipulate.** The search path of §11.5 is precisely what a **virtual environment** alters so the node sees its own dependencies — **Chapter 12**, next.
- **Versions and dependencies are declared, not assumed.** The Python version constraint and every third-party module the import system resolves are declared in `pyproject.toml` and pinned in `poetry.lock` — **Chapter 13**.
- **The GIL and async.** CPython's single-threaded execution of Python code (§11.1) is why the node uses an event loop rather than threads — recalled from **Chapter 2**, realized in **Chapter 16**.
- **The interactive shell.** `hathor-cli shell` drops into the §11.2 REPL with the node loaded — **Chapter 21**.

Recap

Concept	What it is	Why it matters here
CPython	the standard C interpreter	what `hathor-core` runs on
Bytecode / VM	compiled opcodes run by Python's VM	explains `.pyc`, portability, some slowness
REPL	interactive read-eval-print loop	`hathor-cli shell` for live exploration
Module	one `.py` file = a namespace	top-level code runs once → singletons
Package	a directory of modules (`__init__.py`)	dotted name mirrors folder path
`sys.modules`	cache of imported modules	"runs once"; circular-import hazard
`sys.path`	ordered import search path	what virtual environments manipulate

Concept	What it is	Why it matters here
absolute import	full dotted path from root	the dominant, file-revealing style
Why Python	I/O-bound + ecosystem + C hot paths	speed traded for velocity, mitigated

"Python the platform" is the interpreter that turns your source into bytecode and runs it, plus the module-and-import system that turns a directory tree into one navigable program, plus the ecosystem and tooling layered on top. A node accepts Python's performance cost because it is I/O-bound and pushes its hot paths into compiled libraries, buying clarity and a mature ecosystem in return. The single most useful habit to take from this chapter: read every import as a file path, and the whole repository opens up. The next chapter follows the import search path (`sys.path`) to the place packages actually live — and explains what a **virtual environment** really is, the `.venv` you keep seeing but may never have looked inside.

---

1. CPython is the reference implementation of the Python interpreter, written in C. It is what you get from python.org and what runs by default when you type `python`. Alternatives (PyPy, Jython, GraalPy) exist but are not used by `hathor-core`. ↩
2. Bytecode is a compact, low-level instruction set that the Python source is compiled to. It is not your CPU's machine code; it runs on Python's own virtual machine. Cached in `.pyc` files under `__pycache__`. ↩
3. The Python Virtual Machine (PVM) is the part of CPython that executes bytecode — a loop that reads and performs bytecode instructions one at a time. "Virtual machine" here means a software CPU, not a whole virtualized computer. ↩
4. The REPL (Read-Eval-Print Loop) is the interactive Python prompt: it reads a line, evaluates it, prints the result, and loops. Started by running `python` with no arguments. ↩
5. A module is a single `.py` file, importable as a namespace whose attributes are the names defined at its top level. The module's name is the filename without `.py`. ↩
6. A namespace is a mapping from names to objects — a "where names live" container. Modules, packages, classes, and function locals are all namespaces; the dot operator reaches into them. ↩
7. A package is a directory of modules (and sub-packages) treated as one importable unit, traditionally marked by an `__init__.py` file. Its dotted import name mirrors its directory path. ↩
8. `sys.modules` is a dictionary CPython keeps of every module already imported in the current process, keyed by dotted name. Imports check it first, which is why a module's top-level code runs only once. ↩
9. `sys.path` is the ordered list of directories Python searches to find a module on import. It includes the standard library and the active environment's installed-packages directory; the first match wins. ↩